

Flow D - TheHive Case Closure Sync Back into Operations State

Polling recently updated TheHive cases, detecting resolved/closed outcomes, and syncing closure status back into Slack and DataTable

Project done by: [Abdul Rehman \(abdul4rehman215\)](#)

GitHub: [abdul4rehman215](#)

Flow D AWS TheHive Case Closure Sync

Document Type

Workflow project document | Flow D

Prototype Focus

AWS IAM automation | n8n | TheHive 5 | Slack

Project Character

Portfolio prototype with implementation, testing, and validation evidence

Design Goal

Showcase an end-to-end SOC automation workflow with clear business and analyst value

Why this document matters

- This write-up is designed to work both as a project portfolio document and as an implementation-facing prototype note.
- It explains the problem, the workflow design, the operational value to a SOC, and the validation evidence that proves the workflow worked in the lab.
- The screenshots included throughout the document are the actual workflow, node, Slack, DataTable, AWS, and TheHive validation artifacts collected during the build process.

Key outcomes

- Polls TheHive cases on a schedule to find recently updated closed/resolved work items.
- Normalizes the case record and extracts the source reference needed to update the project finding state.
- Pushes the closure result back into Slack and the shared DataTable so the prototype becomes a two-way operating model.

1. Project Summary

Flow D is the closure and feedback workflow of the prototype. Its purpose is to make sure that case outcomes in TheHive do not remain isolated inside the case platform. Instead, the workflow periodically checks for recently updated closed/resolved cases and writes that closure state back into the operational record used by the rest of the project.

This is important because a workflow prototype is much stronger when it can demonstrate a closed loop. With Flow D in place, the system can detect, triage, contain, create alerts and cases, and then synchronize final case status back into the shared operations layer.

2. Problem Statement

Without a feedback loop, investigation and case-closure outcomes stay trapped in the case-management platform. Slack may still show an earlier status, the tracking table may remain open, and future analysis cannot easily distinguish between active and resolved work. That creates operational drift between tools.

Why the problem matters in a SOC

- A SOC needs state consistency across collaboration, tracking, and case platforms.
- Closure status is part of operational truth; if it lives in only one system, analysts lose visibility.
- A feedback loop improves auditability because the final case outcome is preserved in the same dataset as the original alert record.

3. Objective

- Poll TheHive 5 for recently updated cases on a defined cadence.
- Select cases that are recent and whose status indicates closed/resolved handling.
- Normalize each case into a closure record that can be written back into the shared data model.
- Notify Slack and update the DataTable so the lifecycle is visibly closed.

4. Solution Overview

Flow D uses a schedule (and optional manual trigger for testing) to search TheHive cases, filter them by recency and status, and normalize the selected case into a standardized closure object. That object extracts the most important identifiers such as the case ID, case number, title, source reference, assignee, timestamps, and closure summary.

The normalized closure record then feeds two downstream branches: one builds the DataTable update payload, while the other formats a Slack message announcing that the case was closed or resolved. The result is a compact but important workflow that turns TheHive from a sink into a feedback source for the rest of the prototype.

5. Tools and Technologies Used

Tool / Technology	Why it was used in this workflow
TheHive 5 Case Search	Provides the recently updated case data that Flow D inspects for closure state.

n8n schedule / manual trigger	Allows both periodic sync and easy manual validation during build and testing.
JavaScript code nodes	Select recent closed cases, normalize them, and build Slack/DataTable payloads.
Slack	Receives a closure sync message so analysts can see final case state in the team channel.
n8n DataTable	Stores the closure state so the project record reflects resolved case outcomes.

6. How the System Works

Stage	What happens
Scheduled poll	Starts the sync cycle and loads the workflow configuration.
Case search	Queries TheHive for recent cases within the configured lookback window.
Recent closed-case selection	Applies project-source and status logic to find relevant resolved/closed cases.
Normalization	Creates a standard closure record used by downstream steps.
State write-back	Updates DataTable and posts the closure summary to Slack.

Plain-English view

- In plain English, Flow D makes sure the final status of a TheHive case is reflected back in the rest of the prototype.
- It is the project's feedback loop: once a case is closed, the team channel and the tracking table learn about it automatically.

7. Workflow and Architecture Evidence

The workflow below is much smaller than Flows A-C, but it plays an important systems role because it closes the loop. The first figure shows the complete design and the second focuses on the working version after the code-node issues were resolved.



Figure D1. Flow D closure-sync workflow.



Figure D2. Final working Flow D workflow.

The logic layer is where the workflow became stable. The case search result, recent-case selection code, and normalization code together convert a TheHive case into the common closure record used by the project.

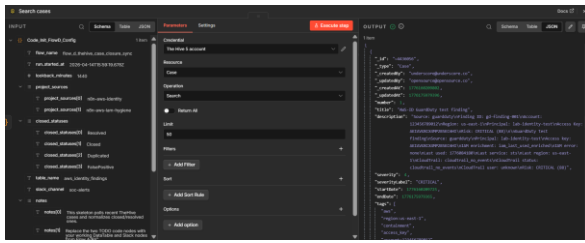


Figure D3. TheHive case search node output.

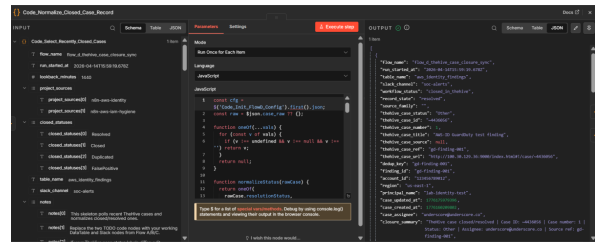


Figure D4. Normalized closed-case record used by downstream nodes.

8. Key Features and USP

- Scheduled closure sync with manual trigger for testing.
- Recency and status-based case selection logic.
- Normalization of case identifiers, timestamps, assignee, and source reference.
- Write-back of closure state into Slack and the shared DataTable.

9. SOC Value and Expected Impact

Flow D improves operational consistency rather than detection speed. Its value is in lifecycle completeness. The rest of the prototype can now show not only how incidents enter the system, but also how investigations and case outcomes flow back into the shared operations view. This reduces state drift and makes reporting cleaner.

Operational area	Expected effect
State consistency	Improved because case outcomes are synchronized across tools.
Analyst visibility	Improved because final case status appears in Slack without manual posting.
Tracking quality	Improved because DataTable reflects the resolved state of handled cases.
Operational maturity	Improved because the prototype demonstrates a full feedback loop rather than a one-way pipeline.

10. Validation and Testing Evidence

The first validation target was proving that the workflow could see the closed case in TheHive and normalize it correctly. The normalized record below shows the extracted fields that drive the downstream outputs.

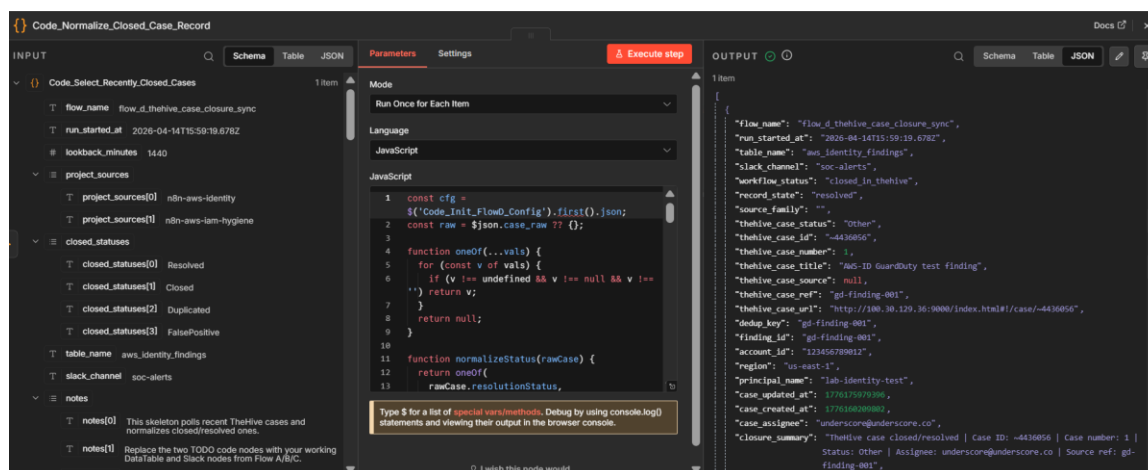


Figure D5. Normalized closed-case record used to update Slack and DataTable.

The final analyst-facing validation is the Slack closure notification, which announces the linked case title, ID, assignee, principal, account, and source reference.

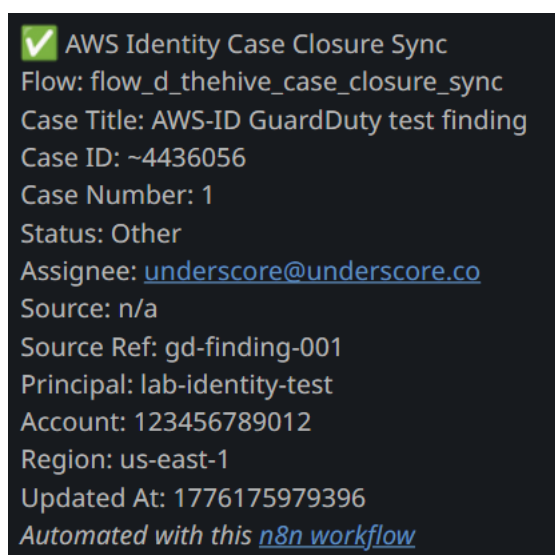


Figure D6. Slack closure notification produced by Flow D.

A closed case visible in TheHive is the source event that Flow D reacts to. The case list below demonstrates the type of recently updated case object that can be synchronized.

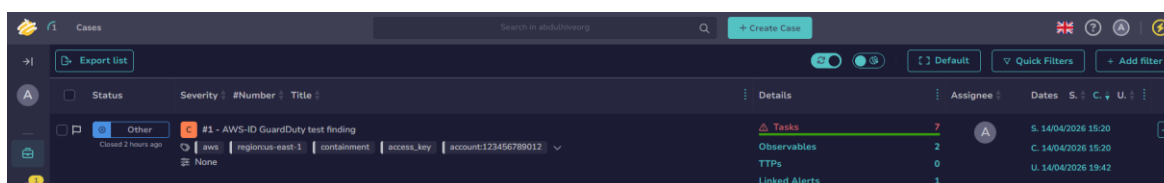


Figure D7. TheHive case list with the closed case that Flow D synchronized.

11. Real-World Use Cases

- Projects that need end-to-end lifecycle demonstration from alert to closure.
- Teams that want shared visibility of case outcomes without manually updating multiple tools.
- Portfolio prototypes showing operational feedback loops and case-state synchronization.

12. Skills Demonstrated

- Case-platform polling and recency logic.
- Normalization of inconsistent multi-tool data into one reusable record shape.
- Lifecycle design thinking: not only detection and response, but final-state synchronization as well.

13. Prototype Limitations

- This prototype uses polling rather than a direct TheHive case-status event trigger.
- The source reference may need extraction fallbacks depending on how the case was created or described.
- Closure-state labels may differ by organization and may need tuning in the config block.

14. Future Enhancements

- Add richer status mapping and closure reason mapping for dashboards or reporting.
- Add bidirectional reconciliation so reopened cases or status reversals are also reflected downstream.
- Replace polling with a more event-driven pattern if a reliable trigger becomes available.

15. Conclusion

Flow D completes the operational loop of the prototype. It proves that the project is not only capable of detecting and responding to issues, but also of synchronizing the final investigation outcome back into the shared operating view. That makes the overall prototype feel significantly more complete and production-minded.

Prepared by **abdul4rehman215**